

Assignment Title:
Linux Fundamentals

Coursework Type:
Individual

Module Name:
ST5003CMD – Operating Systems, Security, And Networks

Submitted By:
Name: Pranisha Bhatta
College ID: 250150

Module Leader:
Name: Rupak Rajbanshi

Table of Content

Table of Content **1**
I. File Operations **2**
II. APT Package Management **3**
III. C Compilation **5**
A. Step 1: Write the Code 5
B. Step 2: Compile and Run 5
C. Step 3: Modify and Recompile 6
IV. Clean and Verification **6**
Conclusion **7**

I. File Operations

```
pranisha@ubuntu: ~/batch38
pranisha@ubuntu:/$ mkdir ~/linux_lab
pranisha@ubuntu:/$ cd ~/linux_lab
pranisha@ubuntu:~/linux_lab$ nano greeting.txt
pranisha@ubuntu:~/linux_lab$ cat greeting.txt
"Hello Linux!"
pranisha@ubuntu:~/linux_lab$ cp greeting.txt backup.txt
pranisha@ubuntu:~/linux_lab$ mv backup.txt old_greeting.txt
pranisha@ubuntu:~/linux_lab$ rm old_greeting.txt
pranisha@ubuntu:~/linux_lab$ mkdir temp_dir
pranisha@ubuntu:~/linux_lab$ rmkdir temp_dir
Command 'rmkdir' not found, did you mean:
  command 'rmdir' from deb coreutils-from-gnu (0.0.0~ubuntu25)
  command 'rmdir' from deb coreutils-from-uutils (0.0.0~ubuntu25)
  command 'rmdir' from deb coreutils-from-busybox (0.0.0~ubuntu25)
  command 'rmdir' from deb coreutils-from-toybox (0.0.0~ubuntu25)
  command 'mmkdir' from deb mblaze (1.1-1build1)
  command 'hmkdir' from deb hfsutils (3.2.6-16ubuntu2)
  command 'mkdir' from deb coreutils-from-gnu (0.0.0~ubuntu25)
  command 'mkdir' from deb coreutils-from-uutils (0.0.0~ubuntu25)
  command 'mkdir' from deb coreutils-from-busybox (0.0.0~ubuntu25)
  command 'mkdir' from deb coreutils-from-toybox (0.0.0~ubuntu25)
Try: sudo apt install <deb name>
pranisha@ubuntu:~/linux_lab$ rm -r temp_dir
pranisha@ubuntu:~/linux_lab$ ls -1 greeting.txt
greeting.txt
```

Fig 1. Essential File Commands

For this part I made a new folder called linux_lab and moved into it using cd. Then I used nano to create a file called greeting.txt and typed "Hello Linux!" inside it. After that I checked the file with cat to make sure it saved properly. I also tried copying and renaming the file using cp and mv, and deleted it after with rm. I made an empty folder and removed it too, just to see how mkdir and rmdir work together.

II. APT Package Management

```
pranisha@ubuntu:~/linux_lab$ sudo apt update
[sudo: authenticate] Password:
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://np.archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://np.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main arm64 Packages [484 kB]
Hit:5 http://np.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:6 http://np.archive.ubuntu.com/ubuntu resolute-updates/main arm64 Packages [612 kB]
Get:7 http://np.archive.ubuntu.com/ubuntu resolute-updates/universe arm64 Packages [260 kB]
Fetched 1,630 kB in 3s (505 kB/s)
13 packages can be upgraded. Run 'apt list --upgradable' to see them.
pranisha@ubuntu:~/linux_lab$ sudo apt install build-essential
Installing:
  build-essential

Installing dependencies:
  bzip2                     gcc-15                    libcrypt-dev            libstdc++-15-dev
  dpkg-dev                  gcc-15-aarch64-linux-gnu  libdpkg-perl            libtsan2
  fakeroot                  gcc-aarch64-linux-gnu     libfakeroot             libubsan1
  g++                       libalgorithm-diff-perl    libfile-fcntllock-perl  lto-disabled-list
  g++-15                    libalgorithm-diff-xs-perl libgcc-15-dev           make
  g++-15-aarch64-linux-gnu  libalgorithm-merge-perl   libhwasan0
  g++-aarch64-linux-gnu     libasan8                  libitm1
  gcc                       libcc1-0                  liblsan0

Suggested packages:
  bzip2-doc                 gcc-15-doc    automake  bison           gdb-aarch64-linux-gnu  libstdc++-15-doc
```

Fig 2. Update the package list

```
pranisha@ubuntu:~/linux_lab$ sudo apt install build-essential
Installing:
  build-essential

Installing dependencies:
  bzip2                     gcc-15                    libcrypt-dev            libstdc++-15-dev
  dpkg-dev                  gcc-15-aarch64-linux-gnu  libdpkg-perl            libtsan2
  fakeroot                  gcc-aarch64-linux-gnu     libfakeroot             libubsan1
  g++                       libalgorithm-diff-perl    libfile-fcntllock-perl  lto-disabled-list
  g++-15                    libalgorithm-diff-xs-perl libgcc-15-dev           make
  g++-15-aarch64-linux-gnu  libalgorithm-merge-perl   libhwasan0
  g++-aarch64-linux-gnu     libasan8                  libitm1
  gcc                       libcc1-0                  liblsan0

Suggested packages:
  bzip2-doc                 gcc-15-doc    automake  bison           gdb-aarch64-linux-gnu  libstdc++-15-doc
  debian-keyring            gcc-multilib  libtool   gcc-doc         git                    make-doc
  debian-tag2upload-keyring autoconf      flex      gcc-15-locales  bzr

Summary:
  Upgrading: 0, Installing: 30, Removing: 0, Not Upgrading: 13
  Download size: 54.3 MB
  Space needed: 194 MB / 13.2 GB available

Continue? [Y/n] Y
Get:1 http://np.archive.ubuntu.com/ubuntu resolute/main arm64 libcc1-0 arm64 16-20260322-1ubuntu1 [51.3 kB]
Get:2 http://np.archive.ubuntu.com/ubuntu resolute/main arm64 libitm1 arm64 16-20260322-1ubuntu1 [27.8 kB]
```

Fig 3. Install the C Compiler (GCC) with build essential package

```
pranisha@ubuntu:~/linux_lab$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Fig 4. Verify installation

```
pranisha@ubuntu:~/linux_lab$ apt search python3
alembic/resolute 1.18.4-1 all
  lightweight database migration tool for SQLAlchemy

antimony/resolute 0.9.3-2.1build2 arm64
  Computer-aided design CAD tool

astro-frameworks/resolute 5.0 all
  Astronomical data analysis frameworks

astro-python3/resolute 5.0 all
  Python 3 packages for astronomy

bamkit/resolute 0.0.1+git20170413.ccd079d-3build1 all
  tools for common BAM file manipulations

beancount/resolute 3.2.0-2build1 all
  Double-entry accounting from text files

berrynet-dashboard/resolute 3.10.2-2build1 all
  deep learning gateway - python3 modules

brltty/resolute,now 6.7-1ubuntu6 arm64 [installed,automatic]
  Access software for a blind person using a braille display

budgie-app-launcher-applet/resolute-updates 2.2.3-1ubuntu1.1 arm64
  Applet to provide an alternative means to launch applications
```

Fig 5. apt search python3

Here I updated the package list first using sudo apt update, since it's good practice to always do this before installing anything new. Then I installed build-essential which includes gcc, the C compiler I needed for the next part. I checked it installed properly by running gcc --version and it showed the version number. I also tried apt search python3 just to see how searching for packages works, even though I didn't install anything from it.

III. C Compilation

A. Step 1: Write the Code

```
 GNU nano 8.7.1                                          hello.c *
#include <stdio.h>

int main() {
        int a=10, b=5;

        printf("Hello, _Linux_Lab!\n");
        printf("Sum_of_%d_and_%d_is_%d\n", a,b,a+b);

        return 0;
}
```

Fig 6. Create Source File and hello.c

For this step I opened nano and wrote a simple C program in a file named hello.c. The program has two integer variables, a and b, and it prints a greeting message plus the sum of the two numbers using printf. It was pretty straightforward once I got the syntax right.

B. Step 2: Compile and Run

After writing the code I compiled it using gcc -Wall hello.c -o hello_program. The -Wall part just turns on extra warnings so if there's anything wrong with the code gcc will let me know. Then I checked the executable was actually created with ls -l, and ran it using ./hello_program. It printed the greeting and showed the sum as 15, which is correct since a was 10 and b was 5.

```
pranisha@ubuntu:~/linux_lab$ nano hello.c
pranisha@ubuntu:~/linux_lab$ gcc hello.c -o hello_program
pranisha@ubuntu:~/linux_lab$ ls -l hello_program
-rwxrwxr-x 1 pranisha pranisha 70496 Sep  8 21:52 hello_program
pranisha@ubuntu:~/linux_lab$ ./hello_program
Hello, _Linux_Lab!
Sum_of_10_and_5_is_15
pranisha@ubuntu:~/linux_lab$ gcc -Wall hello.c -o hello_program_warn
pranisha@ubuntu:~/linux_lab$ ./hello_program_warn
Hello, _Linux_Lab!
Sum_of_10_and_5_is_15
```

Fig 7. Compile and Execute

C. Step 3: Modify and Recompile

```
pranisha@ubuntu:~/linux_lab$ nano hello.c
pranisha@ubuntu:~/linux_lab$ gcc -Wall hello.c -o hello_program
pranisha@ubuntu:~/linux_lab$ ./hello_program
Hello, _Linux_Lab!
Sum_of_200_and_5_is_205
```

Fig 8. Iterative Development

I went back into the file and changed the value of a from 10 to 200. But just changing the code wasn't enough — I had to recompile it again using the same gcc command, otherwise the program would still run the old version. After recompiling and running it again, it showed the new sum as 205. This part actually helped me understand that the compiled program and the source file are two separate things, so any change needs a recompile before it shows up.

IV. Clean and Verification

```
pranisha@ubuntu:~/linux_lab$ cd ~
pranisha@ubuntu:~$ ls -lh ~/linux_lab/
total 40K
-rw-rw-r-- 1 pranisha pranisha  15 Sep  8 08:56 greeting.txt
-rw-rw-r-- 1 pranisha pranisha 144 Sep  8 22:46 hello.c
-rwxrwxr-x 1 pranisha pranisha 69K Sep  8 22:47 hello_program
-rwxrwxr-x 1 pranisha pranisha 69K Sep  8 21:54 hello_program_warn
pranisha@ubuntu:~$ cat ~/linux_lab/hello.c
#include <stdio.h>

int main() {
        int a=200, b=5;

        printf("Hello, _Linux_Lab!\n");
        printf("Sum_of_%d_and_%d_is_%d\n", a,b,a+b);

        return 0;
}
pranisha@ubuntu:~$ rm -rf ~/linux_lab
```

Fig 9. Final Checks

For the last part I went back to my home directory using cd, then listed everything inside linux_lab using ls -lh to see the file sizes properly. I also used cat one more time to check the final version of my hello.c file and make sure everything was saved the way I wanted.

Conclusion

Overall this lab helped me get more comfortable with using the Linux terminal for basic tasks. I practiced creating, copying, renaming and deleting files, and also got a better understanding of file permissions using ls -l. The APT section showed me how package management works on Linux, especially how important it is to update before installing anything. The C programming part was probably the most useful for me — writing, compiling and running a program, and then seeing what happens when I change the code but forget to recompile. That really made it clear that the source code and the compiled program are two different things, and one doesn't update automatically when the other changes. By the end of the lab I felt a lot more confident moving around the terminal and understood the basic flow of writing and running C programs on Linux.